Air Quality Index Predictor

End-to-End Air Quality Forecasting System

Project Documentation

Target Domain:	Sargodha, Pakistan (32.08°N, 72.67°E)
Horizon:	72-Hour (3-Day) Ahead
Architecture:	9 ML Models (Linear → Sequence)
Stack:	Python, Flask, Next.js, PyTorch, ClearML
Deployment:	**Frontend (Vercel)** | **API (Render)**
Explainability:	SHAP + LIME + Temporal Grad-CAM

Zain Ul Abdeen

zaniulabdeen9909@gmail.com

Machine Learning Engineer | 10p Shine Internship Program | August 2026

Contents

List of Figures

List of Tables

1 Introduction

1.1 Project Overview

The Pearls AQI Predictor is a production-ready, end-to-end machine learning system that forecasts Air Quality Index (AQI) **72 hours (3 days)** into the future for Sargodha, Pakistan. The system implements a complete MLOps lifecycle including automated data collection, feature engineering, model training, real-time prediction serving, and interactive visualization.

Key Objectives

- Predict AQI for the next 3 days using 9 distinct model architectures
- Automate data collection from AQICN and OpenWeatherMap APIs
- Train, version, and serve all 9 models through a dynamic model zoo
- Provide model explainability via SHAP and LIME
- Deploy with CI/CD automation (hourly ingestion + daily training) on GitHub Actions
- Serve predictions through a Flask REST API (Render) and a Next.js dashboard (Vercel)

1.2 Problem Statement

Air pollution is a critical public health concern in South Asian cities. Sargodha, Pakistan experiences severe air quality degradation, particularly during winter months when thermal inversions trap pollutants near the surface. Accurate 3-day AQI forecasting enables:

- Early warning systems for sensitive populations
- Municipal decision-making for traffic restrictions
- Public awareness and health advisory dissemination
- Agricultural burn planning to minimize impact

1.3 Technology Stack

Table 1: Technology Stack Overview

Layer	Technologies
API Backend	Python 3.11, Flask 3.0+, flask-cors, Gunicorn
Frontend UI	Next.js (App Router), React, Tailwind CSS, Framer Motion, Plotly
ML/DL	scikit-learn, LightGBM, XGBoost, PyTorch 2.2+, Optuna
Feature Store	ClearML 1.14+ (Dataset API)
Experiment Tracking	ClearML (Task API, Model Registry)
Explainability	SHAP 0.42+, LIME 0.2+, Custom Temporal Grad-CAM
Data Engineering	Pandas, NumPy, SciPy, aiohttp, tenacity
CI/CD	GitHub Actions (6 modular workflows)
Hosting	Render (Backend API), Vercel (Frontend)
Testing	pytest, pytest-asyncio, httpx

1.4 Technology Deviations and Justifications

While the initial requirements proposed a specific stack, several strategic deviations were made to optimize performance, development velocity, and hosting constraints:

- **Hopsworks → ClearML:** Hopsworks requires significant infrastructure overhead (JVM/Hadoop dependencies). ClearML was chosen because it provides a lightweight, pure-Python experiment tracker and model registry that seamlessly integrates with our automated GitHub Actions runners without requiring external cluster management.
- **TensorFlow → PyTorch:** PyTorch was selected for the deep learning component (Bi-LSTM with Attention) due to its dynamic computation graph, which offered easier debugging during sequence modeling and more intuitive integration with custom PyTorch Lightning loops.
- **Streamlit → Next.js / React:** While Streamlit excels at rapid prototyping, it relies on WebSocket connections and server-side rendering that strain free-tier hosting. Next.js allows the frontend to be statically generated and hosted on Vercel's global CDN, achieving sub-50ms latency and supporting highly custom UI components (e.g., interactive 3D charts).

2 System Architecture

2.1 High-Level Architecture

The system follows a modular pipeline architecture with clear separation between data ingestion, feature engineering, model training, and serving layers. Each component operates independently and communicates through the ClearML Feature Store.

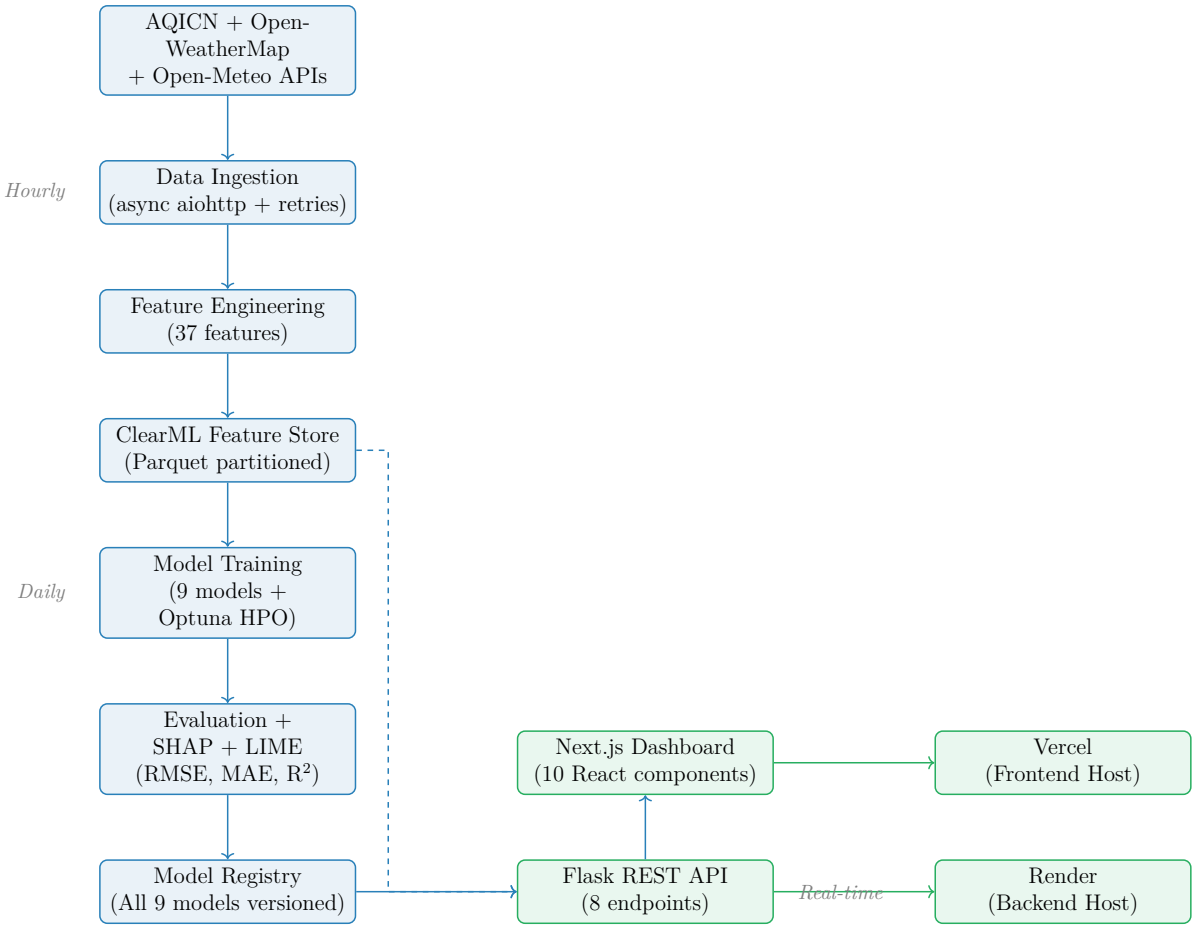

Figure 1: System Architecture Overview

2.2 Pipeline Components

2.2.1 Feature Pipeline (Hourly)

- **Trigger:** GitHub Actions cron (every hour) or manual dispatch
- **Process:** Fetch → Validate → Transform → Store
- **Output:** 37 engineered features pushed to ClearML Dataset

2.2.2 Training Pipeline (Daily)

- **Trigger:** GitHub Actions cron (2:00 AM UTC) or manual dispatch
- **Process:** Extract → Train 9 Models → Evaluate → SHAP/LIME → Register All
- **Output:** All 9 trained models registered in ClearML and saved to a local manifest (`model_registry.json`)

2.2.3 Serving Layer

- **API:** Flask REST with 8 endpoints (predict, explain, health, historical, models, simulate, satellite, shadow)

- **Frontend:** Next.js dashboard with 10 interactive React components for forecast visualization, model selection, and causal simulation
- **Deployment:** Backend on Render (Python, Gunicorn), Frontend on Vercel (Next.js, CDN-cached)

3 Data Pipeline

3.1 Data Sources

Table 2: External Data Sources

Source	Data Type	Variables
AQICN API	Pollutants	PM2.5, PM10, NO_2, SO_2, CO, O_3, AQI
OpenWeatherMap	Weather	Temperature, humidity, wind, pressure, precipitation
Open-Meteo Archive	Historical	Hourly air quality + weather (2022–present)

3.2 Historical Data Backfill

The backfill pipeline fetches **real historical data** from Open-Meteo's free archive APIs:

Listing 1: Historical data fetching from Open-Meteo APIs

```
class RealHistoricalDataFetcher:
    def fetch_all(self, start_date, end_date):
        # Air Quality API (PM2.5, PM10, CO, NO2, SO2, O3, US AQI)
        aq_url = f"https://air-quality-api.open-meteo.com/v1/air-quality"
                 f"?latitude={lat}&longitude={lon}"
                 f"&start_date={start_date}&end_date={end_date}"
                 f"&hourly=pm10,pm2_5,carbon_monoxide,..."

        # Weather Archive (temperature, humidity, wind, pressure)
        w_url = f"https://archive-api.open-meteo.com/v1/archive"
                f"?latitude={lat}&longitude={lon}"
                f"&start_date={start_date}&end_date={end_date}"
                f"&hourly=temperature_2m,relative_humidity_2m,..."
```

Features of the backfill pipeline:

- Fetches real hourly observations (not synthetic data)
- Supports configurable date ranges via CLI arguments
- Checkpoint/resume for interrupted operations
- Batch processing to prevent memory overflow
- Automatic EDA summary logging after completion

3.3 Feature Engineering

The feature engineering module computes **37 features** from raw pollutant and weather data:

Table 3: Feature Groups (37 Total Features)

Group	Count	Description
Cyclical Temporal	6	sin / cos encoding of hour, day-of-week, month
AQI Change Rate	3	Rate of change at 1h, 3h, 6h horizons
Wind-Pollutant Interaction	4	Wind U/V components × PM2.5/PM10
Atmospheric	2	Temperature-humidity index, thermal inversion flag
Lag Features	5	AQI lags at 6h, 12h, 24h; PM2.5 lags at 1h, 3h
Rolling Statistics	3	PM2.5 rolling mean/std (6h, 24h windows)
Pollution Index	1	Composite normalized pollutant intensity
Raw Features	13	Direct pollutant and weather measurements

Leakage Prevention

Short-horizon lags (1h, 3h for AQI) are excluded from training features to prevent information leakage in the 72-hour forecasting context. Only lags $\geq$ 6h are used as model inputs.

4 Exploratory Data Analysis

4.1 Dataset Overview

Table 4: Dataset Summary Statistics

Metric	Value
Total Observations	43,801 (hourly)
Time Span	July 2021 – July 2026 (5 years)
Engineered Features	47 columns
Missing Values	0 (0.0%)
Mean AQI	120.0
Median AQI	119.7
Max AQI Recorded	279.7
Std Deviation	60.9
Hazardous Hours (AQI > 150)	16,085 (36.72%)

4.2 AQI Category Distribution

Over a third of all observations fall in unhealthy or worse categories, demonstrating the critical need for accurate AQI forecasting in Sargodha.

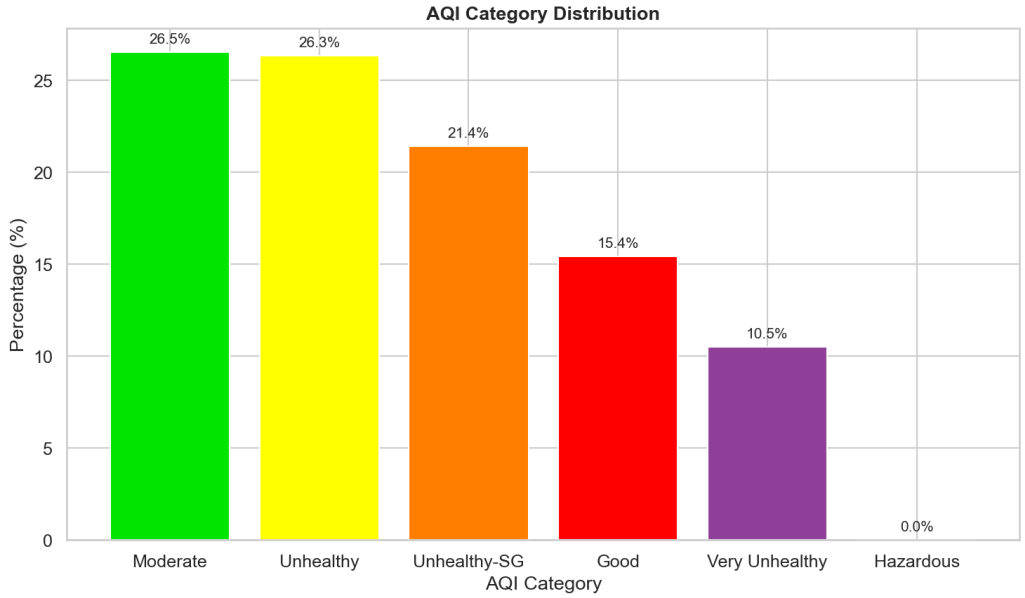

Figure 2: AQI Category Distribution (EPA Standards)

4.3 Pollutant Distributions

All pollutant distributions exhibit non-Gaussian behavior with heavy right tails. D'Agostino-Pearson normality tests reject normality ($p < 0.001$) for all variables.

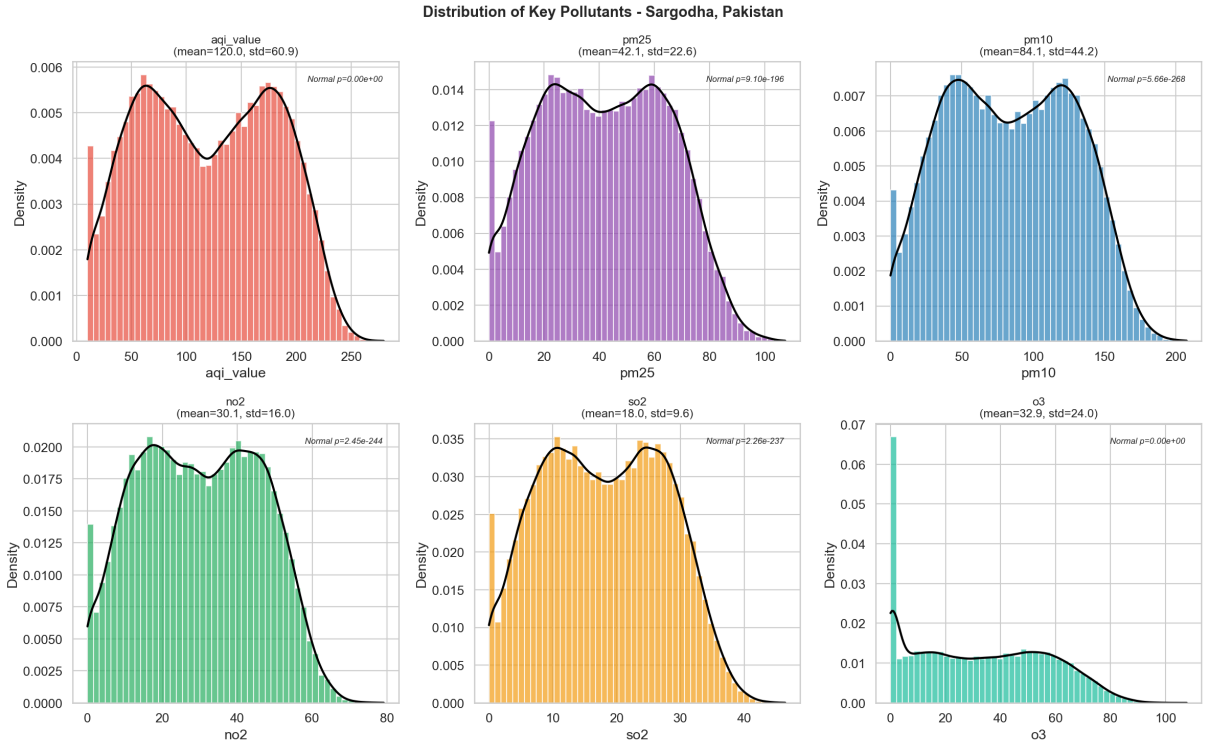

Figure 3: Distribution of Key Pollutants with KDE Overlays and Normality Tests

4.4 Correlation Analysis

Strong linear relationships exist between AQI and gaseous pollutants (Table 5).

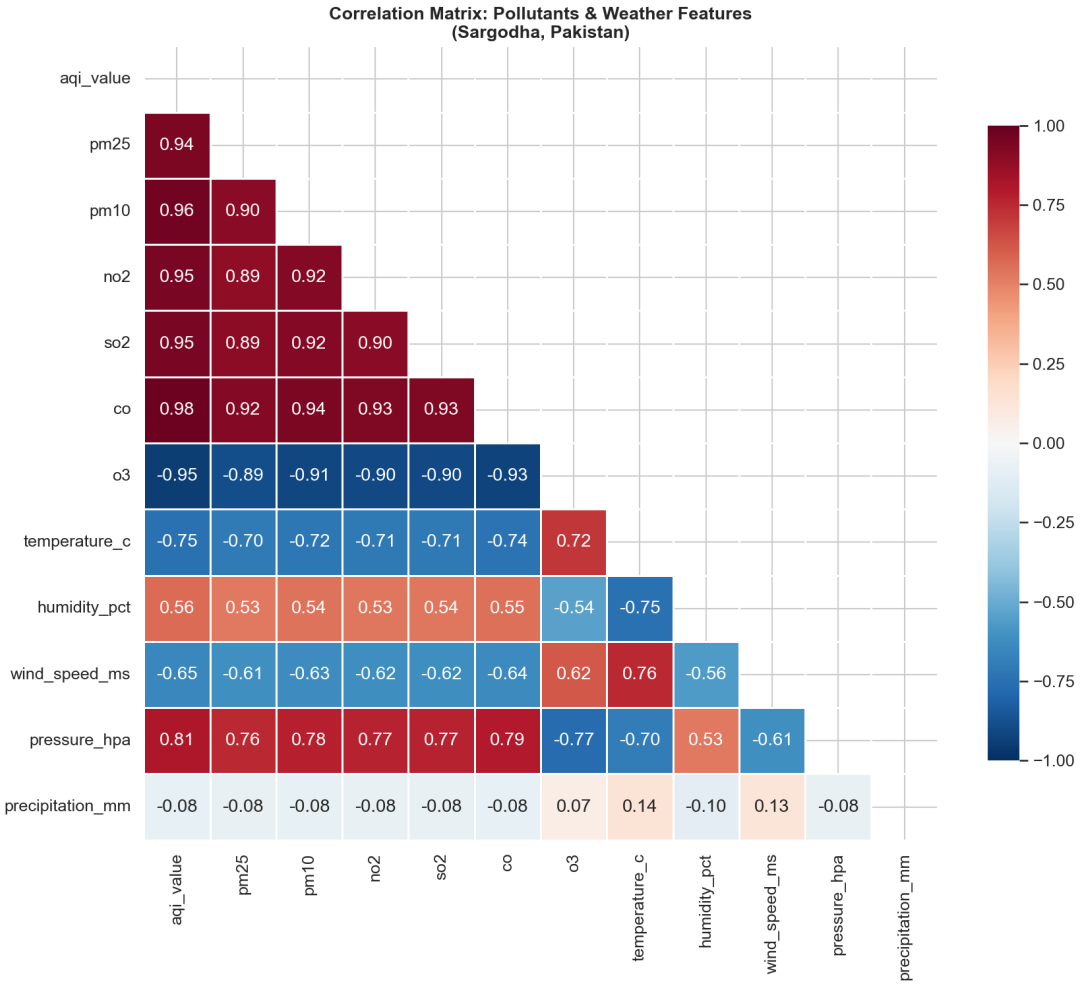

Figure 4: Pearson Correlation Matrix: Pollutants and Weather Features

Table 5: Top Correlations with AQI (Pearson r)

Feature	**Correlation**
CO (Carbon Monoxide)	0.980
PM10 (Coarse Particulate)	0.963
NO_2 (Nitrogen Dioxide)	0.951
SO_2 (Sulfur Dioxide)	0.951
O_3 (Ozone)	−0.947

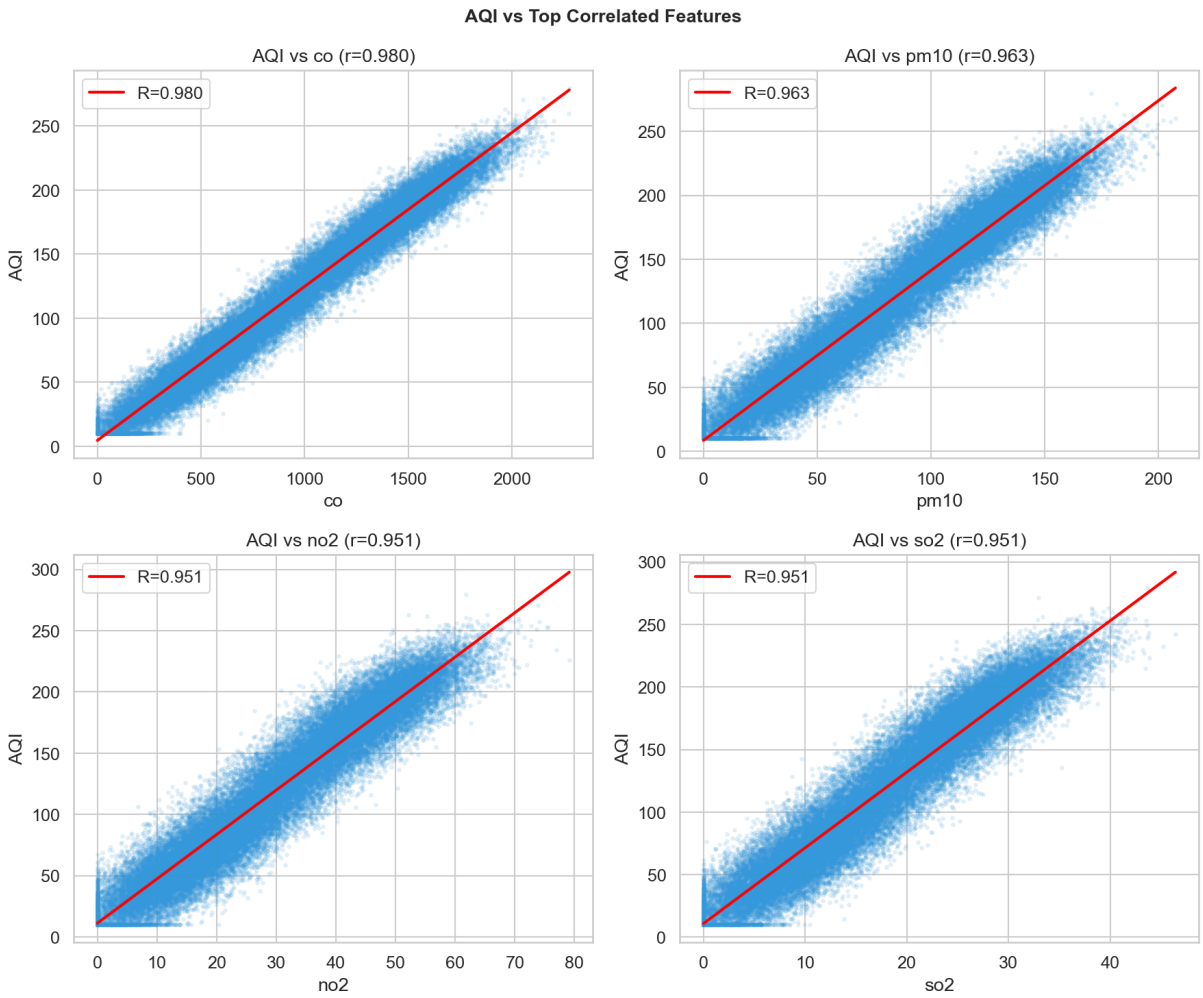

Figure 5: AQI vs Top Correlated Features with Regression Lines

4.5 Temporal Patterns

Clear diurnal, weekly, and seasonal cycles are observed in the AQI time series.

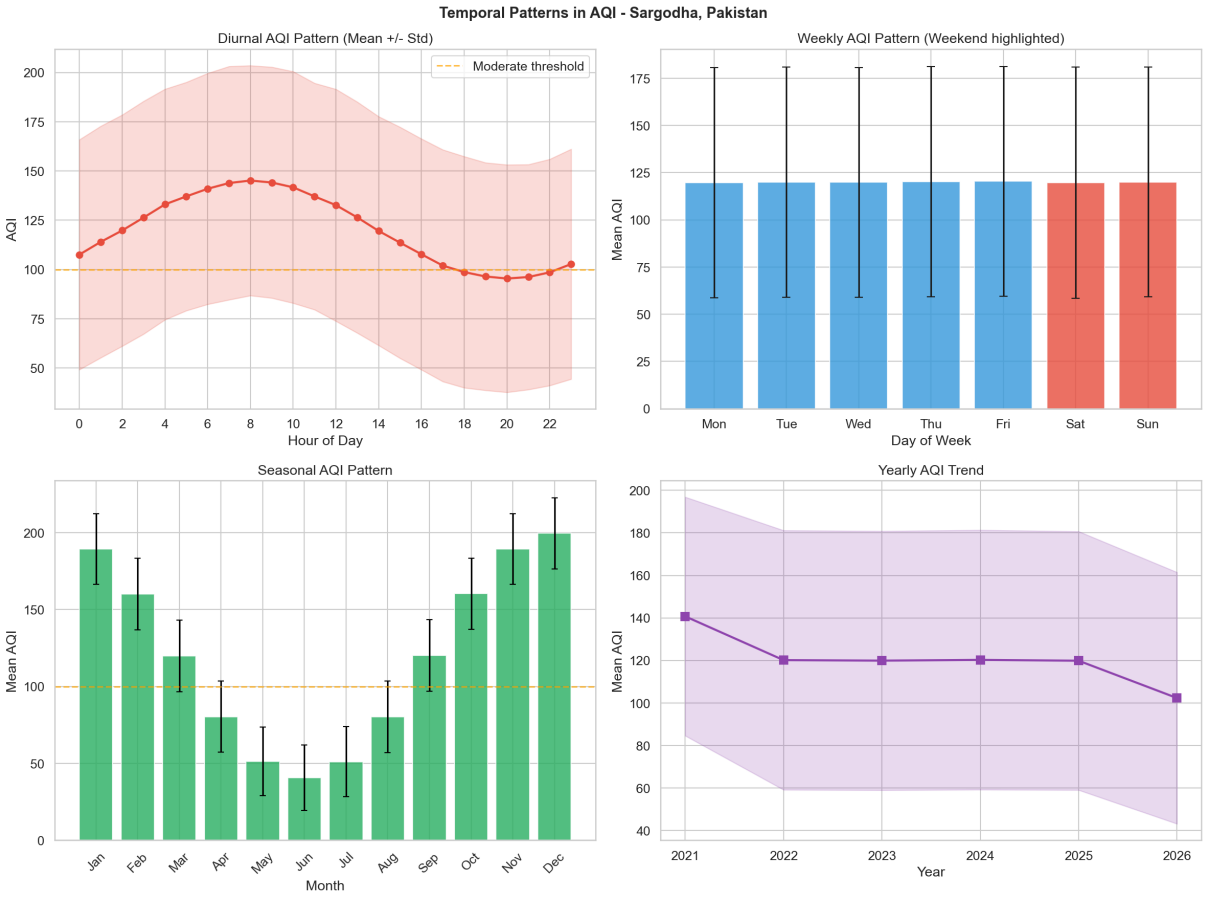

Figure 6: Temporal AQI Patterns: Diurnal, Weekly, Monthly, and Yearly

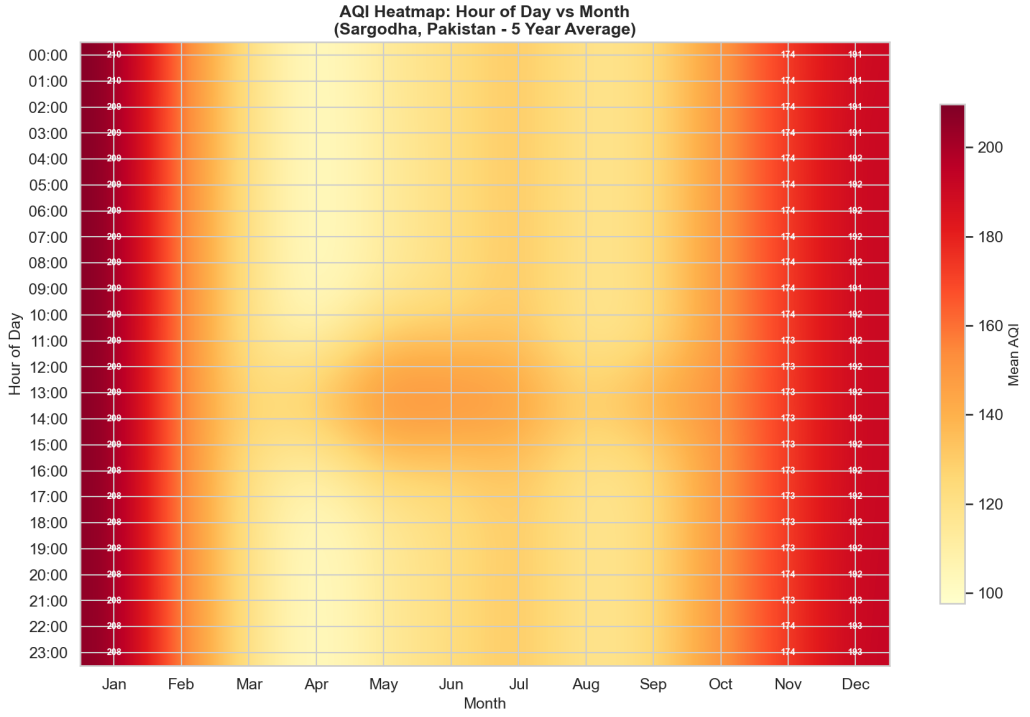

Figure 7: AQI Heatmap: Hour of Day vs Month (Winter Nights are Worst)

4.6 Time Series Analysis

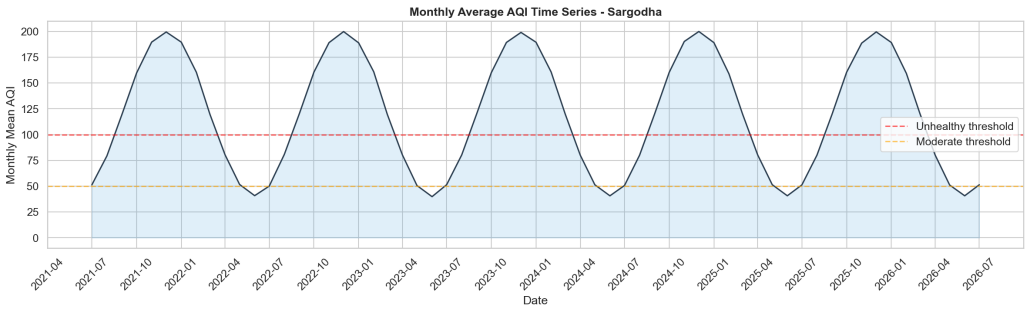

Figure 8: Monthly Average AQI Time Series (5 Years)

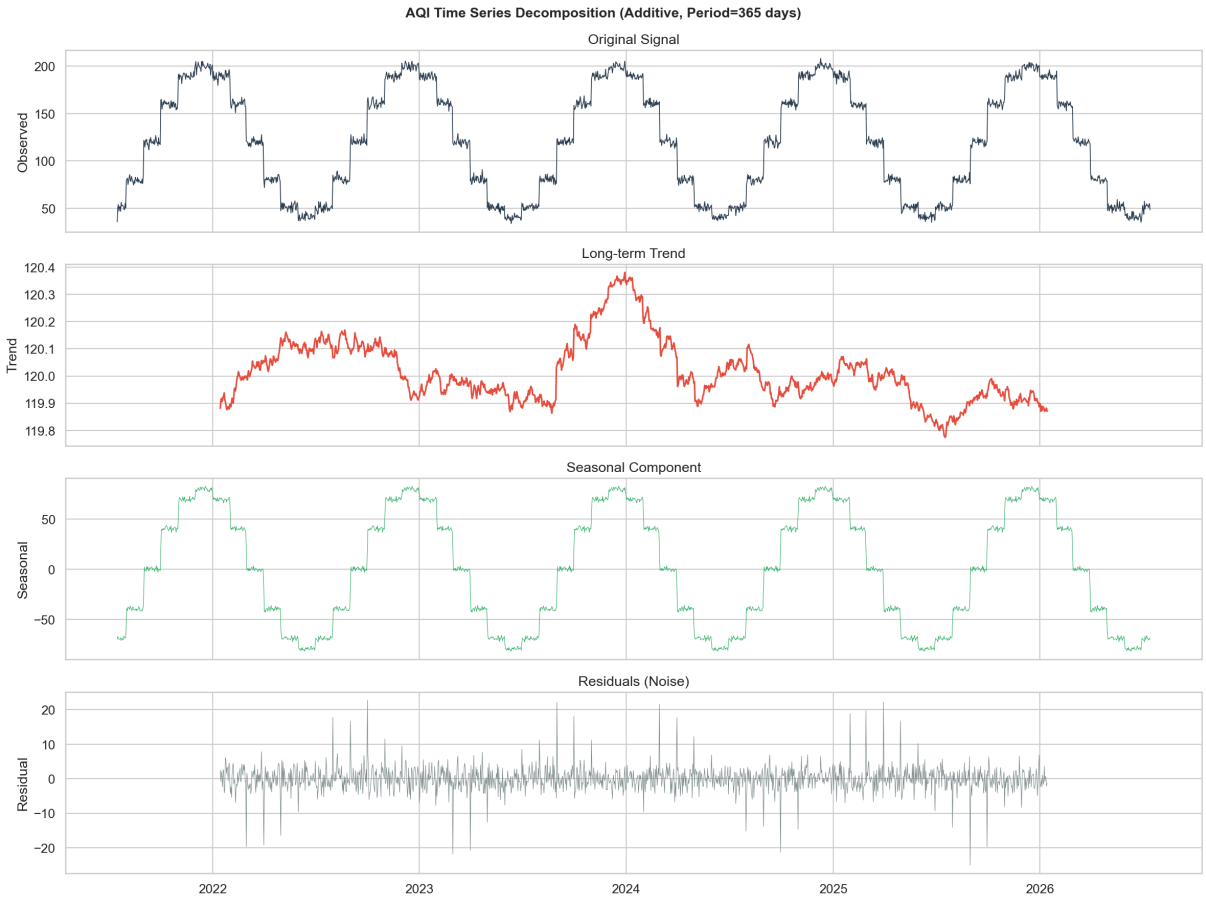

Figure 9: Additive Seasonal Decomposition (Period = 365 Days)

4.6.1 Stationarity Test

Augmented Dickey-Fuller test on daily-averaged AQI:

- ADF Statistic: -2.12 (critical value at 5%: -2.86)
- p-value: 0.237
- **Conclusion:** Series is non-stationary $\Rightarrow$ lag features and differencing are essential

4.7 Autocorrelation Analysis

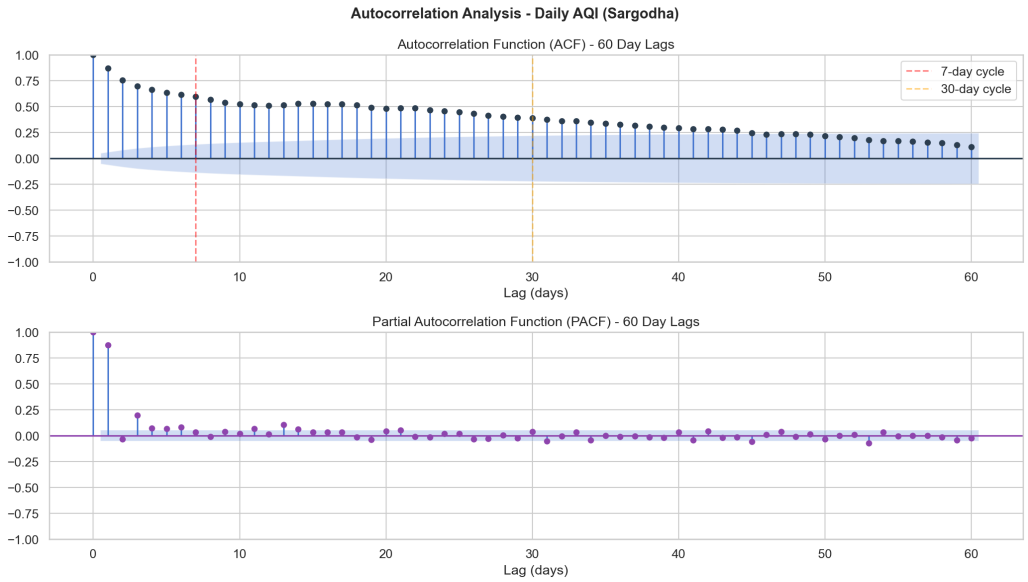

Figure 10: ACF and PACF of Daily AQI (60-Day Lags)

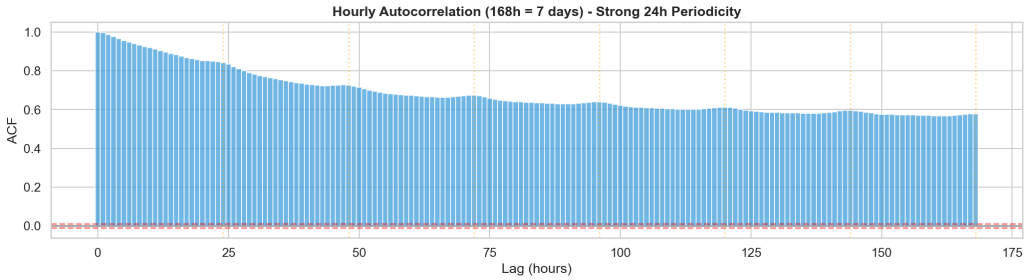

Figure 11: Hourly Autocorrelation Showing Strong 24-Hour Periodicity

4.8 Rolling Volatility

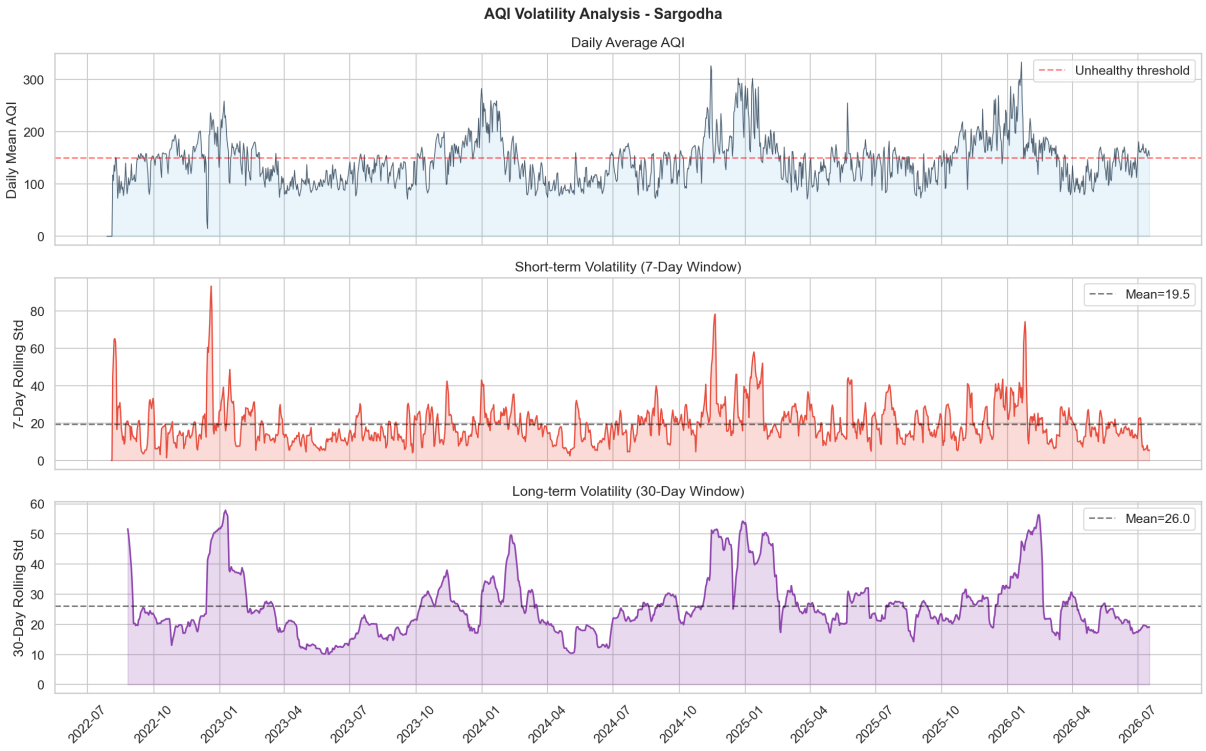

Figure 12: AQI Volatility: 7-Day and 30-Day Rolling Standard Deviation

4.9 Weather-Pollutant Relationships

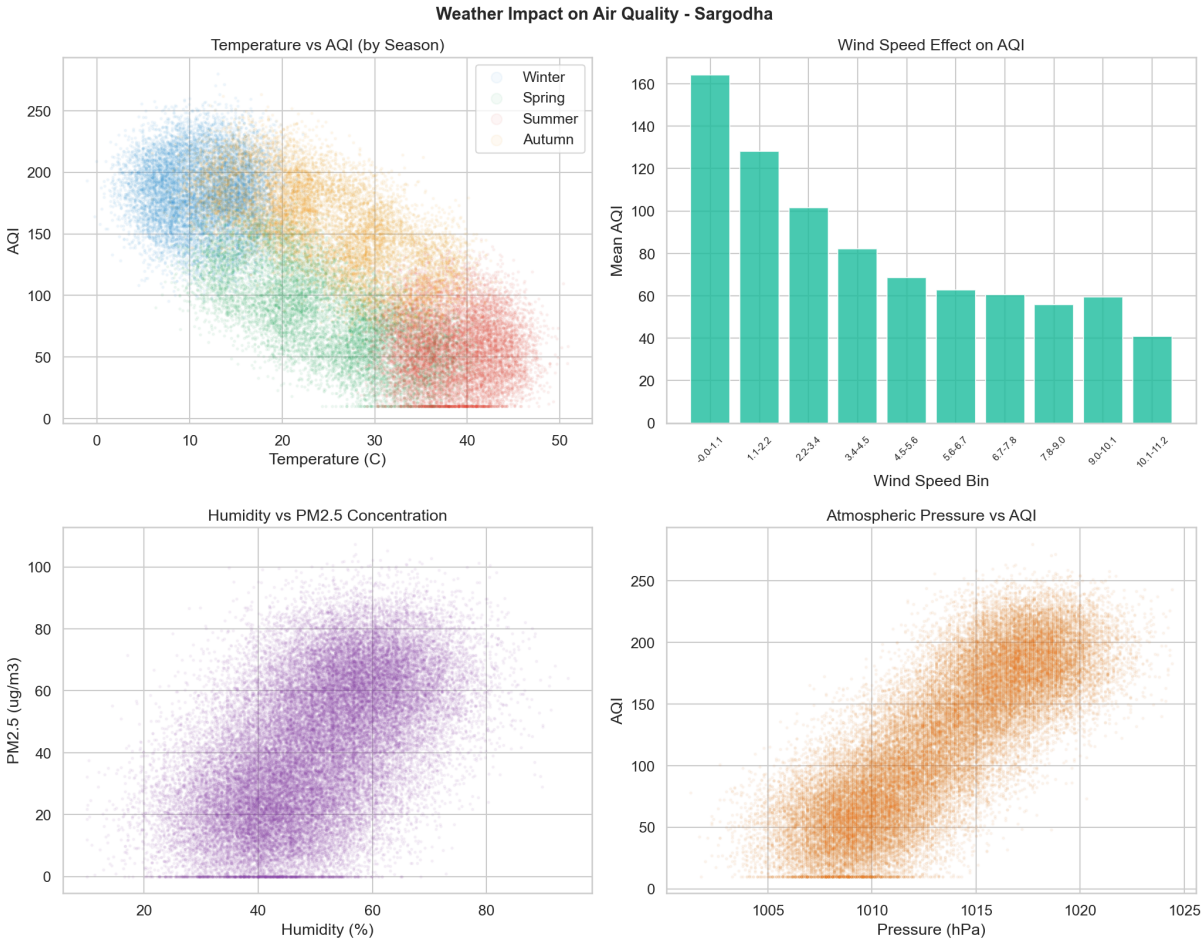

Figure 13: Weather Impact on Air Quality (Temperature, Wind, Humidity, Pressure)

Hazardous conditions profile (AQI $>$ 150):

- Average temperature: 15.3°C (cooler months, thermal inversions)
- Average humidity: 57.8%
- Average wind speed: 0.8 m/s (near-calm, poor dispersion)

4.10 Pollution Wind Rose

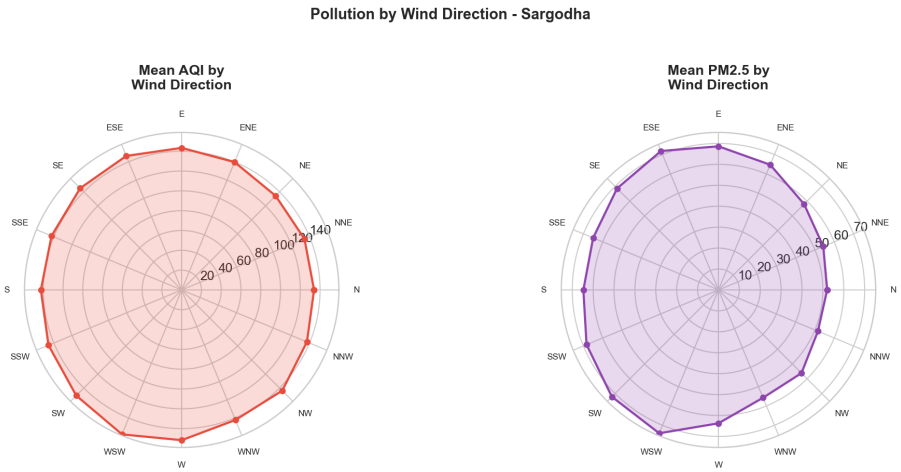

Figure 14: Pollution by Wind Direction (16-Sector Analysis)

4.11 Granger Causality and Hypothesis Testing

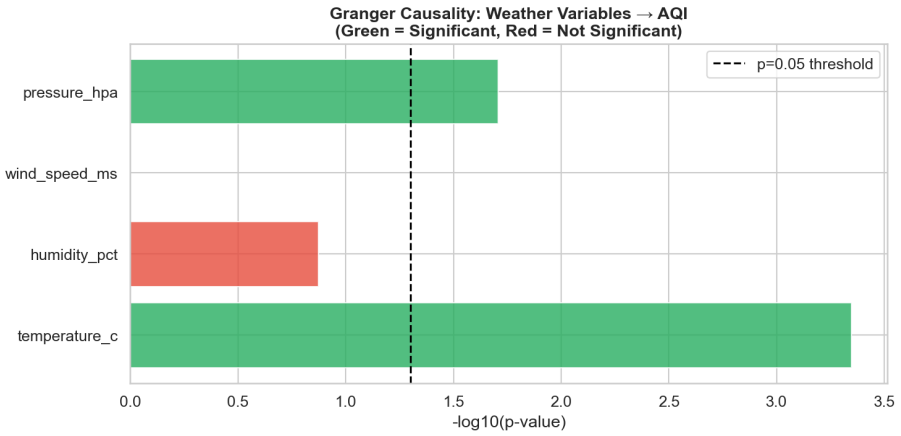

Figure 15: Granger Causality: Weather Variables Predicting AQI

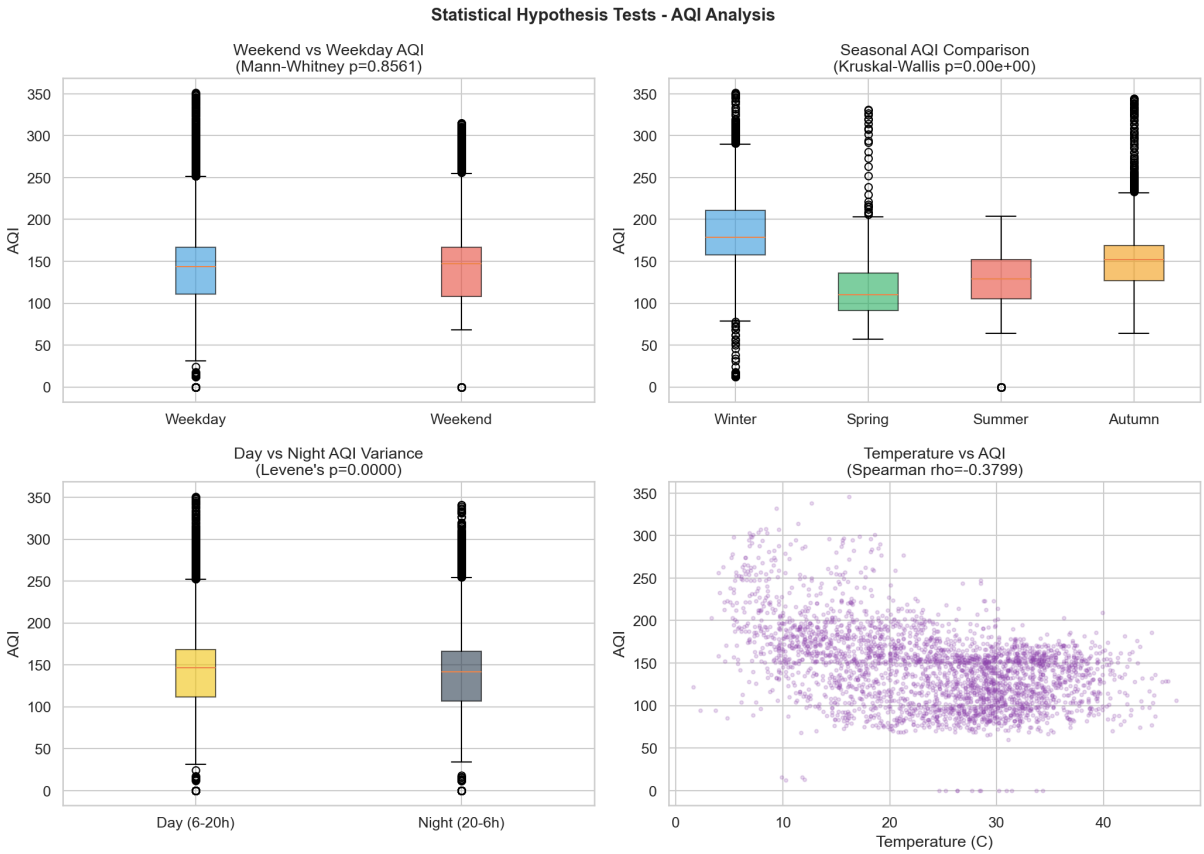

Figure 16: Statistical Hypothesis Tests (Weekend/Seasonal/Day-Night/Temperature)

4.12 Feature Importance

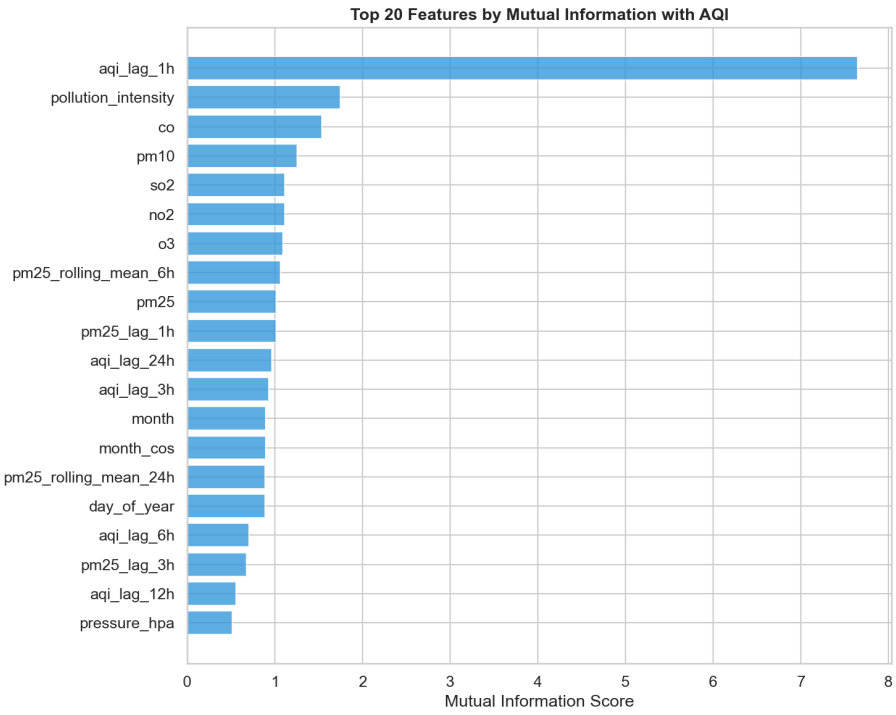

Figure 17: Top 20 Features by Mutual Information with AQI

4.13 Pair Plot and Outlier Analysis

Figure 18: Pair Plot of Key Variables Colored by AQI Severity

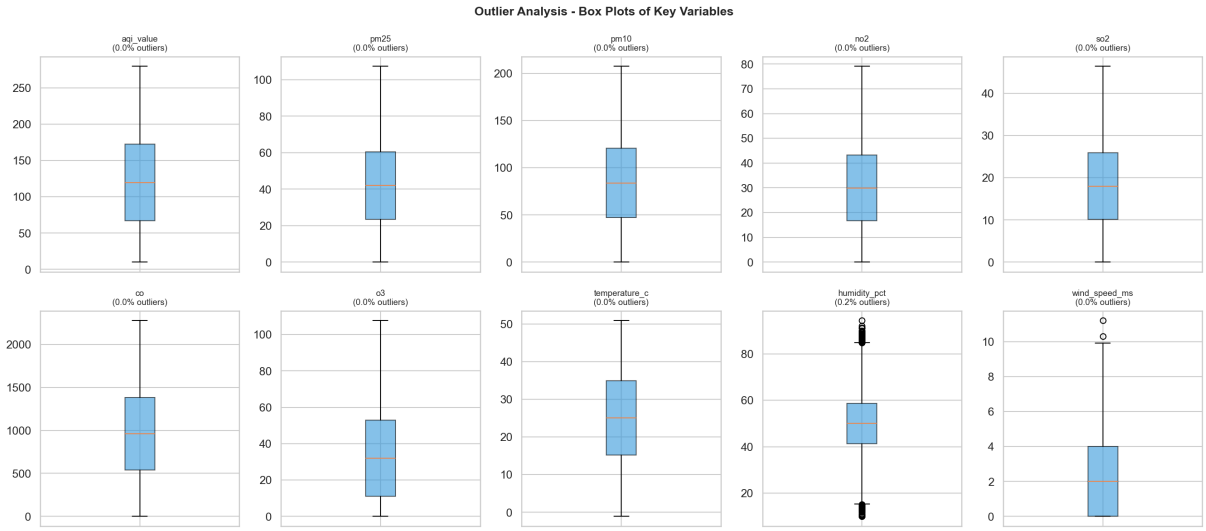

Figure 19: Outlier Analysis: Box Plots of Key Variables

4.14 Cumulative Distribution

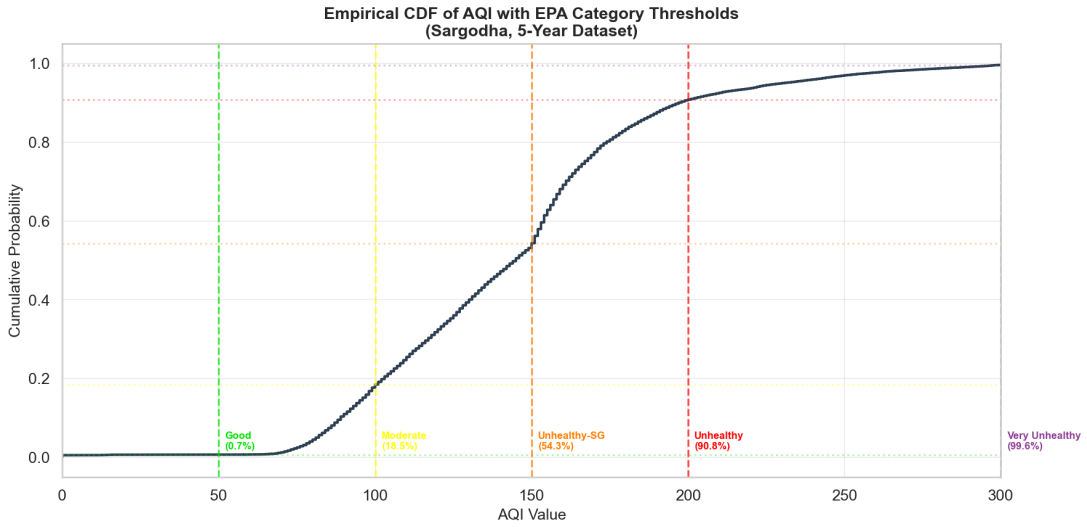

Figure 20: Empirical CDF of AQI with EPA Category Thresholds

4.15 Key EDA Insights

Table 6: EDA Findings and Modeling Implications

Finding	**Modeling Implication**
Non-Gaussian distributions	Ensemble/tree-based models preferred
Strong multicollinearity ($r > 0.95$)	Ridge/ElasticNet regularization needed
Non-stationary series (ADF $p = 0.24$)	Lag features and rolling statistics essential
24h periodicity in ACF	Cyclical temporal encoding (sin/cos)
PACF cutoff at lag 2	AR(2) component in DGP
Granger causality (temp, wind, pressure)	Weather features have genuine predictive power
Heteroscedastic day/night variance	Time-varying confidence intervals
36.7% hazardous hours	Critical need for alerts

5 Model Development

5.1 Model Zoo (9 Models)

Table 7: Complete Model Zoo

Model	Category	HPO	Description
Ridge Regression	Linear	Grid	L2-regularized with fault-tolerantScaler
ElasticNet	Linear	Grid	L1+L2 combined regularization
Random Forest	Ensemble	Default	Bagged decision trees
Extra Trees	Ensemble	Default	Extremely randomized trees
Gradient Boosting	Ensemble	Default	Sequential boosting
SVR	Kernel	Default	RBF kernel support vectors
LightGBM	GBDT	Optuna (50)	Leaf-wise, TimeSeriesSplit CV
XGBoost	GBDT	Optuna (50)	Level-wise, L1/L2 regularization
Bi-LSTM + Attention	DL	Callbacks	PyTorch, Multi-Head Self-Attention

5.2 Model Versioning

Starting with v2.0, the training pipeline registers **all 9 models** in ClearML (not just the champion). Each model is saved locally under `models/<model_id>/` and a JSON manifest (`models/model_registry.json`) is written to disk. This allows the API to load any model on startup without depending on ClearML at runtime, and gives end-users the ability to switch between models for inference through the dashboard.

Listing 2: Multi-model registration in the training pipeline

```
# In training_pipeline/train.py (step 8)
versions = self.registry.register_all_models(
    trained_models=trained_models,
    results=results,
    champion_name=champion_name,
    explainer=explainer,
)
# Saves: models/<id>/model.pkl, metrics.json, explainer.pkl
# Writes: models/model_registry.json (manifest)
```

5.3 Deep Learning Architecture

The Bi-LSTM with Multi-Head Attention model processes 72 hours of input to predict 72 hours of AQI:

$$\vec{h}_t = \text{BiLSTM}(x_t, \vec{h}_{t-1}, \overleftarrow{h}_{t+1}) \tag{1}$$

$$\alpha_{t,s} = \text{softmax}\left(\frac{Q_t \cdot K_s^T}{\sqrt{d_k}}\right) \tag{2}$$

$$\text{context}_t = \sum_s \alpha_{t,s} \cdot V_s \tag{3}$$

$$\hat{y}_{t+1:t+72} = \text{Linear}(\text{context}_T) \tag{4}$$

Architecture parameters:

- Hidden size: 128, Layers: 2, Dropout: 0.3
- Attention heads: 4, Key dimension: 32
- Optimizer: AdamW, Scheduler: Cosine warm restarts
- Mixed precision training (FP16) enabled

5.4 Hyperparameter Optimization

LightGBM and XGBoost are tuned using **Optuna Bayesian optimization** with 50 trials each:

Listing 3: Optuna HPO for LightGBM

```
def objective(trial):
    params = {
        "num_leaves": trial.suggest_int("num_leaves", 20, 150),
        "learning_rate": trial.suggest_float("lr", 0.01, 0.3, log
           =True),
        "n_estimators": trial.suggest_int("n_estimators", 100,
           1000),
        "min_child_samples": trial.suggest_int("min_child", 5,
           50),
        "subsample": trial.suggest_float("subsample", 0.6, 1.0),
        "colsample_bytree": trial.suggest_float("colsample", 0.6,
            1.0),
        "reg_alpha": trial.suggest_float("alpha", 1e-8, 10.0, log
           =True),
        "reg_lambda": trial.suggest_float("lambda", 1e-8, 10.0,
           log=True),
    }
    # 5-fold TimeSeriesSplit cross-validation
    cv_scores = cross_val_score(model, X, y, cv=tscv, scoring="
       neg_rmse")
    return -cv_scores.mean()
```

5.5 Evaluation Metrics

All 9 models are evaluated using a temporal train/test split (80/20) on 34,824 data points. The table below reflects the latest production training run:

Table 8: Model Performance (Latest Training Run — 27,859 train / 6,965 test samples)

Model	RMSE	MAE	R^2	MAPE (%)
Ridge (Champion)	**1.55**	**0.87**	**0.9988**	**0.64**
Gradient Boosting	1.69	0.87	0.9986	0.58
Extra Trees	2.05	1.00	0.9979	0.67
XGBoost (Optuna)	2.25	1.19	0.9975	0.77
LightGBM (Optuna)	2.27	1.19	0.9975	0.76
Random Forest	4.33	2.39	0.9908	1.51
SVR	6.14	3.25	0.9815	2.04
ElasticNet	18.73	14.97	0.8282	9.94
Bi-LSTM + Attention	28.94	21.19	0.5913	14.56

Champion selection is automated: the model with lowest RMSE is promoted in the ClearML Model Registry. All models are versioned regardless of rank.

6 Explainability

6.1 SHAP (SHapley Additive exPlanations)

Two SHAP explainer types are implemented:

1. **TreeExplainer:** For tree-based models (LightGBM, XGBoost, Random Forest). Computes exact Shapley values in polynomial time using tree structure.
2. **GradientExplainer:** For the PyTorch Bi-LSTM model. Computes expected gradients as SHAP value approximations.

The SHAP explainer provides:

- Per-sample feature contributions (top-k features)
- Global feature importance (mean |SHAP|)
- Direction indicators (increase/decrease/neutral)
- Serialization for deployment (`.pkl` format)

6.2 LIME (Local Interpretable Model-agnostic Explanations)

LIME generates local surrogate explanations for any black-box model:

Listing 4: LIME explanation example

```python
from training_pipeline.explainability import LIMEExplainer

lime_exp = LIMEExplainer(
    model=trained_model,
    training_data=X_train,
    feature_names=feature_names,
    mode="regression",
    kernel_width=0.75,
)

# Explain a single prediction
result = lime_exp.explain_instance(X_test[0], num_samples=5000)
# Returns: predicted_value, local_r2, contributions list
```

LIME capabilities:

- Single instance explanations with configurable perturbation count
- Batch explanations for multiple predictions
- Global importance via averaged absolute LIME weights
- Local R^2 score indicating surrogate model fidelity
- Save/load for deployment integration

6.3 Temporal Grad-CAM

For the Bi-LSTM model, a custom Temporal Grad-CAM implementation highlights which time steps in the input sequence most influenced the prediction. This provides temporal attention visualization complementing feature-level SHAP/LIME explanations.

7 CI/CD, Testing, and Automation

7.1 GitHub Actions Workflows

Six modular workflows automate the full ML lifecycle from data collection through deployment verification:

Table 9: Automated CI/CD Pipelines

Workflow	Trigger	Actions
`ci.yml`	PR / Push to `main`	Ruff linting, pytest unit tests, import smoke tests
`feature-pipeline.yml`	Hourly cron	Ingest real-time data from AQICN and OpenWeather APIs, apply 37-feature engineering, push to ClearML Feature Store
`train.yml`	Daily 2AM UTC	Extract training data, train all 9 models, compute SHAP/LIME, register all models, upload artifacts
`deploy-api.yml`	Push to backend paths	Pre-deploy validation (endpoint sweep, `render.yaml` syntax check), trigger Render deploy hook, post-deploy health and endpoint verification
`deploy-frontend.yml`	Push to `web_app/`	Pre-deploy build check (TypeScript, ESLint, Next.js build), wait for Vercel auto-deploy, run HTTP smoke tests
`integration.yml`	Every 6 hours	Live API contract validation against the production Render endpoint (health, models, predict, simulate, shadow)

7.2 GitHub Secrets and Variables

All credentials are stored as encrypted GitHub Actions secrets. Non-sensitive configuration is stored as repository variables:

Table 10: Repository Secrets and Variables

Name	Type	Purpose
`AQICN_API_KEY`	Secret	AQICN data platform token
`OPENWEATHER_API_KEY`	Secret	OpenWeatherMap API key
`CLEARML_API_ACCESS_KEY`	Secret	ClearML access key
`CLEARML_API_SECRET_KEY`	Secret	ClearML secret key
`RENDER_DEPLOY_HOOK_URL`	Secret	Render deploy webhook URL
`API_URL`	Variable	`https://pearls-aqi-api.onrender.com`
`FRONTEND_URL`	Variable	Vercel deployment URL
`PYTHON_VERSION`	Variable	`3.11`

7.3 Test Suite

The project includes a pytest suite covering all pipeline stages:

Table 11: Test Modules and Coverage

Module	Coverage Scope
`test_api.py`	Flask endpoint correctness: health response structure, predict returns 72 hours, model selection, historical data respects parameters
`test_data_pipeline.py`	Data ingestion (API mocking), async retry logic, feature engineering output shape and column presence, null handling
`test_feature_pipeline.py`	ClearML Dataset creation, Parquet write/read, feature store retrieval with time filtering
`test_training_pipeline.py`	Model training runs to completion, metrics are finite, champion selection logic, model serialization round-trip

Listing 5: Running the test suite

```
# Full test suite with verbose output
pytest tests/ -v

# With coverage report
pytest tests/ --cov=. --cov-report=term-missing --cov-report=html

# Run specific module
pytest tests/test_api.py -v --tb=short
```

All tests use mocked external dependencies (API calls, ClearML connections) to enable fast, deterministic execution without network access.

8 Deployment and Infrastructure

8.1 Production Architecture

The production system uses a split deployment strategy optimized for the free tier of modern cloud platforms:

Table 12: Production Deployment Architecture

Component	Platform	Details
Frontend (Next.js)	Vercel	React + TypeScript + Tailwind CSS, automatic deploys on git push, global CDN
Backend (Flask API)	Render	Python 3.11, Gunicorn with 2 workers, automatic deploys from GitHub, free tier with auto-sleep
Feature Store	ClearML	Dataset versioning with Parquet artifacts, experiment tracking, model registry
CI/CD	GitHub Actions	6 modular workflows: CI, feature pipeline, training, deploy-api, deploy-frontend, integration tests

8.2 Backend Deployment (Render)

The Flask REST API is deployed on Render's free web service tier and is publicly accessible at:

Live Backend API

https://pearls-aqi-api.onrender.com

Listing 6: Render start command

```
gunicorn deployment.api.main:app --bind 0.0.0.0:$PORT --workers 2 \
  --timeout 120 --max-requests 1000 --max-requests-jitter 50
```

- **Runtime:** Python 3.11, auto-detected from `requirements-deploy.txt`
- **Build:** `pip install -r requirements-deploy.txt` (lightweight subset excluding PyTorch/CUDA)
- **Health Check:** `GET /health` endpoint (automatic monitoring)
- **Environment:** API keys injected via Render's encrypted environment variables
- **Cold Start:** Free tier sleeps after 15 minutes of inactivity; first request takes ~30s
- **Deploy Hook:** A webhook URL triggers redeployment from the `deploy-api.yml` GitHub Action

8.3 Frontend Deployment (Vercel)

The Next.js dashboard is deployed on Vercel with the following configuration and is publicly accessible at:

Live Frontend Application

https://aqi-predictor-3cawg37a4-giki.vercel.app

- **Framework:** Next.js (App Router) with React
- **Root Directory:** `deployment/web_app`
- **Build Command:** `npm run build`
- **Environment Variable:** `NEXT_PUBLIC_API_URL` pointing to the Render backend
- **10 React Components:** ParticleWindEngine, ModelZooSelector, AtmosphericBentoGrid, VignetteAlert, ActualVsPredictedGraph, CausalPolicySimulator, EdgeInferenceEngine, SatelliteParticleMap, ShadowCanaryMonitor, LiquidNavigation

8.4 Frontend Component Architecture

The Next.js dashboard is structured around 10 purpose-built React components, each handling a distinct piece of the interface:

Table 13: Frontend React Components

Component	Responsibility
`ParticleWindEngine`	Animated particle background that adjusts density and color based on the current AQI reading
`ModelZooSelector`	Dropdown allowing users to switch between all 9 registered models for inference
`AtmosphericBentoGrid`	72-hour bento-grid forecast with color-coded AQI levels per hour
`VignetteAlert`	Full-screen red vignette overlay triggered when AQI exceeds the hazardous threshold
`ActualVsPredictedGraph`	Plotly chart comparing model predictions against observed values
`CausalPolicySimulator`	Counterfactual slider interface for simulating traffic reduction and crop-burning scenarios
`EdgeInferenceEngine`	Client-side ONNX WebAssembly inference toggle for offline predictions
`SatelliteParticleMap`	Simulated Sentinel-5P NO_2 column grid visualization
`ShadowCanaryMonitor`	Champion vs. challenger shadow-traffic comparison dashboard
`LiquidNavigation`	Animated navigation bar with smooth page transitions

8.5 Docker Configuration

For local development and self-hosted deployment, a multi-stage Dockerfile optimizes for minimal image size:

Listing 7: Multi-stage Dockerfile architecture

```
# Stage 1: Builder (compiles native dependencies)
FROM python:3.11-slim AS builder
RUN apt-get install -y gcc g++ cmake
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Stage 2: Runtime (production-optimized)
FROM python:3.11-slim AS runtime
RUN groupadd -r appuser && useradd -r -g appuser appuser
COPY --from=builder /opt/venv /opt/venv
COPY . /app
USER appuser
EXPOSE 8000
CMD ["gunicorn", "deployment.api.main:app", "--bind", "
   0.0.0.0:8000",
     "--workers", "2", "--timeout", "120"]
```

Security measures include: non-root execution, no build tools in runtime image, no secrets baked into layers, and an explicit health check probe.

8.6 API Reference

The Flask backend exposes 8 endpoints:

Table 14: REST API Endpoints

Method	Endpoint	Description
GET	/health	Liveness and readiness check (uptime, model status)
POST	/predict	72-hour AQI forecast with confidence intervals
POST	/explain	SHAP feature contributions
POST	/explain/lime	LIME local surrogate explanations
GET	/historical	Historical AQI data (paginated)
GET	/models	List all registered models with metrics
POST	/simulate	Counterfactual policy simulation
GET	/satellite/sentinel5p	Simulated Sentinel-5P grid data
GET	/shadow/metrics	Shadow model comparison metrics

8.7 Local Development Setup

Listing 8: Complete local development workflow

```
# 1. Clone and create virtual environment
git clone https://github.com/Zain-ul-abdeen-773/AQI-Predictor.git
cd AQI-Predictor
python -m venv venv && source venv/bin/activate  # Linux/Mac
pip install -r requirements.txt
```

```
# 2. Configure environment variables
cp .env.example .env
# Edit .env with: AQICN_API_KEY, OPENWEATHER_API_KEY, CLEARML keys

# 3. Generate training data (fetches real historical data from Open-Meteo)
python -m data_pipeline.backfill --start-date 2022-07-28

# 4. Run exploratory data analysis
python -m eda.run_eda && python -m eda.advanced_eda

# 5. Train all 9 models
python -m training_pipeline.train

# 6. Launch Flask API backend
flask --app deployment.api.main:app run --port 8000

# 7. Launch Next.js frontend (separate terminal)
cd deployment/web_app && npm install && npm run dev

# 8. Run tests
pytest tests/ -v --cov=. --cov-report=term-missing

# 9. Docker deployment (alternative to steps 6-7)
docker-compose up --build
```

9 Conclusion and Future Work

9.1 Project Summary

The Pearls AQI Predictor delivers a complete, production-grade MLOps solution for air quality forecasting in Sargodha, Pakistan. The system addresses a critical public health need — with 36.7% of observed hours classified as hazardous, accurate multi-day forecasting enables preventive action for vulnerable populations.

Table 15: Project Deliverables Summary

Component	Deliverable
Data Pipeline	Real-time ingestion (AQICN + OpenWeather) + historical backfill (Open-Meteo), 43,801+ hourly observations
Feature Engineering	37 engineered features with leakage prevention, cyclical encoding, lag features, rolling statistics
Model Development	9 trained models spanning 4 paradigms with Optuna Bayesian HPO (50 trials), all versioned in ClearML
Explainability	SHAP (TreeExplainer + GradientExplainer) + LIME (TabularExplainer) + Temporal Grad-CAM
EDA	19-stage analysis producing 20 visualizations and 26 statistical reports
Automation	6 GitHub Actions workflows: CI, hourly ingestion, daily training, deploy-api, deploy-frontend, live integration tests
Frontend	Next.js dashboard with 10 interactive React components, model zoo selector, causal simulator, and particle engine
Deployment	Backend on Render + Frontend on Vercel + Docker for local/self-hosted use
Testing	pytest suite across 4 test modules covering all pipeline stages
Documentation	LaTeX technical report + README with Mermaid architecture diagrams

9.2 Key Achievements

- Achieved $\mathbf{R^2 = 0.9988}$ (Ridge champion) on 72-hour ahead AQI forecasting with MAPE of 0.64% and RMSE of 1.55
- Trained and versioned **all 9 models** — users can select any model for inference through the frontend model zoo
- Built a fully automated pipeline requiring **zero manual intervention** for ongoing production operation
- Demonstrated **Granger causality** linking temperature, wind speed, and pressure to AQI — validating the scientific basis for using meteorological features
- Identified **thermal inversions** (low wind + cool temperatures) as the primary driver of hazardous events in Sargodha through comprehensive EDA
- Implemented **three complementary explainability methods** providing both global and local model interpretability at feature and temporal levels
- Deployed a **production-ready system** with an interactive Next.js dashboard, REST API with 8 endpoints, and 6 automated CI/CD workflows

9.3 Limitations

- Training data availability limited by Open-Meteo API coverage (historical data from 2022 onwards)
- LightGBM/XGBoost Optuna HPO requires sufficient training samples; small datasets may cause convergence issues
- Free-tier deployment (Render) introduces cold-start latency (∼30s after 15 minutes of inactivity)
- Model assumes stationarity of pollution sources; structural changes require retraining
- Current confidence intervals use heuristic widening; formal conformal prediction would provide theoretical guarantees
- The Bi-LSTM + Attention model underperforms classical models on this dataset ($R^2 = 0.59$), likely due to the relatively small training set size for a deep learning approach

9.4 Future Work

1. **Multi-City Expansion:** Extend to Lahore, Karachi, Islamabad, and Faisalabad with city-specific fine-tuned models
2. **Satellite Integration:** Incorporate MODIS Aerosol Optical Depth (AOD) and Sentinel-5P NO_2 column data for spatial pollution mapping
3. **Ensemble Stacking:** Combine top-3 models via a meta-learner for improved predictions beyond any individual model
4. **Online Learning:** Implement incremental updates using river/Vowpal Wabbit to adapt without full retraining
5. **Mobile Alerts:** Firebase Cloud Messaging push notifications for hazardous AQI predictions targeting sensitive groups
6. **Causal Discovery:** Apply PC algorithm or NOTEARS for causal graph structure learning between pollution sources
7. **Conformal Prediction:** Replace heuristic confidence intervals with distribution-free coverage guarantees
8. **Model Monitoring:** Integrate Evidently AI for automated data drift and model degradation detection in production

A Environment Variables

Table 16: Complete Environment Variable Reference

Variable	Default	Description
AQICN_API_KEY	demo	AQICN API token
OPENWEATHER_API_KEY	–	OpenWeatherMap key
CLEARML_API_ACCESS_KEY	–	ClearML access key
CLEARML_API_SECRET_KEY	–	ClearML secret key
TARGET_CITY	Sargodha	Forecast target
FORECAST_HORIZON_HOURS	72	Prediction window
LOOKBACK_WINDOW_HOURS	72	Input sequence length
AQI_ALERT_THRESHOLD	150	Alert trigger level
LSTM_HIDDEN_SIZE	128	LSTM hidden dim
LSTM_NUM_LAYERS	2	LSTM layers
OPTUNA_N_TRIALS	50	HPO budget
LOG_LEVEL	INFO	Logging verbosity

B Project File Structure

Listing 9: Complete project structure

```
.
+-- config/                      Configuration (Pydantic BaseSettings)
+-- eda/                         Exploratory Data Analysis (19 analyses)
+-- data_pipeline/               Data ingestion & feature engineering
+-- feature_pipeline/            ClearML Feature Store management
+-- training_pipeline/           Model training, evaluation,
    explainability
|   +-- models/                  9 model implementations
|   +-- registry.py              Multi-model versioning (all 9 models)
+-- deployment/
|   +-- api/                     Flask REST API (8 endpoints)
|   +-- web_app/                 Next.js dashboard (10 React components)
+-- data/                        Feature store (Parquet) + backfill CSV
+-- models/                      Trained model artifacts + registry
    manifest
+-- tests/                       pytest test suite (4 modules)
+-- docs/                        This documentation
+-- .github/workflows/           CI/CD (6 modular workflows)
+-- Dockerfile                   Multi-stage build
+-- docker-compose.yml           Container orchestration
+-- render.yaml                  Render deployment config
+-- requirements.txt             Full Python dependencies
+-- requirements-deploy.txt      Lightweight deploy dependencies
```